Reflection:

Abstract:

For the implementation of the Tiny Mart assignment I used the Go programming language. Go is a statically typed, compiled language designed by Google. Go is known for its simplicity and performance. Some of the use cases for this language are building web servers, networking tools, and command line applications. Wheather Go is an object oriented language is sometimes disputed but what seems to be the most popular opinion is that it does fall under the OOP category but it deviates from the traditional OOP paradigm.

Approach:

I first began by creating a UML diagram and mapping where the classes should inherit from. When designing this diagram I had the mindset of writing this program a C++ style OOP program but since Go's OOP paradigm deviates slightly I had to adjust my method in actual development. In Go there are no classes, just structs and the way inheritance works is by utilizing composition which is the embedding of structs into one another. Polymorphism is handled by interfaces which are essentially a collection of method signatures. Any struct you define can implement an interface as long as it is public and that said structs method signatures match the interfaces signatures. I created a base Product interface to capture the common functionality of all product types. For inheritance I created the base product struct and embedded that into the audio product, video product, and book product struct. The same was done for the ebook and paper book, they were essentially just structs and what was embedded into them was the book struct. From there it was relatively easy just implementing the getters and setters along with the helper functions.

Key Learnings:

I learned that Go does not have traditional classes or inheritance. Instead all of that is handled by composition and interfaces. I also learned that export rules of Go, unlike C++ there are no private or public keywords to identify which member variables or methods are accessible by other parts of the code. This is determined by whether the names started with a capital or a lowercase letter, this was also true for interfaces. I also learned the file structure of go program, since go uses packages (pre built Go code) similar to C++ libraries when first initializing a project you initialize a go.mod file which tracks the dependencies your code uses, and their versions and other information about build requirements or constraints. Also I learned that for a project like this which emulates or tries to the conventional OOP paradigm it is best to split the program into files based on each class/struct you are creating.

Unique Distinctive Features:

Interfaces which I discussed earlier were a bit confusing to me at first but it is a feature that I really enjoyed and liked about this language so much easier than C++. The composition was something that was very unique to me because it makes inheritance feel like an

interchangeable building block as opposed to rigided family tree modules that C++ uses. For someone who tries to avoid inheritance as much as possible I really enjoyed this feature. Also the exporting of member variables and methods was very unique since it was all based on naming conventions. Slices are a big feature that were very powerful in my opinion. A slice in Go is a flexible, dynamically-sized view into an array that behaves like a lightweight list structure. In this project, I used a slice to store pointers to Product interface types in the Cart struct. This allowed the cart to hold a mix of different product types (AudioProduct, VideoProduct, etc.) and dynamically grow as items were added. Since the slice stored pointers referencing the underlying structs, I could call each product's methods (like DisplayProdInfo) polymorphically at runtime, leveraging Go's interface and method resolution system.

Likes and Dislikes:

I really enjoyed the simplicity of this language, there is not a lot of overly complicated syntax and it is quite readable even without much experience. I really enjoyed composition. This way of inheritance is very easy for me to understand and work with and I can see where this would be better than a more rigid approach like C++ has. Interfaces are another feature I really enjoyed since they give more flexibility. What I did not enjoy was the exporting of member variables and methods being decided by the capitalization of the first letter in a name. This seems like it would cause a lot of errors quickly since it's such an easy thing to miss.

Additional Notes:

I did encounter challenges with this assignment since it did not follow the OOP paradigm that is traditionally implemented by languages like C++ and that was a very big learning curve. To overcome that I did have to do a lot of Googling and had to leverage AI. After completing this assignment though I feel as if this language is very simple and when needing a statically typed compiled language I might gravitate to this as opposed to C++ just because of the ease of writing it.

Conclusion:

Overall, implementing Tiny Mart in Go was a good experience that helped me understand some of the concepts behind the language. It challenged my understanding of object-oriented design and forced me to think more about composition and interface-driven development. Since this language focuses on simplicity, understanding the concepts and how to actually do something is not as difficult as it is in C++. This definitely made me realize how language design shapes the way we can solve a problem.